JWT

A JWT (JSON Web Token) is a compact, URL-safe standard (RFC 7519) used in backend engineering to securely transmit information between parties as a JSON object. It is most commonly used for stateless authentication and authorization in modern web apps, APIs, and microservices.

Anatomy of a JWT

A JWT consists of three Base64URL-encoded parts separated by dots (`.`):

1. **Header:** Metadata describing the token type (JWT) and the cryptographic signing algorithm used (such as HS256 or RS256).
2. **Payload (Claims):** The actual data or statements about the user or subject (e.g., `sub` for user ID, `role` for permissions, `exp` for expiration time). The payload is not encrypted, only encoded. Anyone who intercepts the token can decode and read it.
3. **Signature:** A cryptographic hash created by combining the encoded header, payload, and a server secret key (or private key). It ensures tamper-evidence—if anyone alters the data inside the header or payload, the signature becomes invalid.

How JWT Authentication Works

1. **Login:** The user sends their credentials (username and password) to the backend server.
2. **Issuance:** The server verifies the credentials and generates a signed JWT containing user claims.
3. **Storage:** The server sends the token back to the client, which stores it (typically in memory, local storage, or an HttpOnly cookie).
4. **Request:** On every subsequent request to a protected endpoint, the client attaches the JWT in the HTTP header (usually as `Authorization: Bearer <token>`).

5. **Verification:** The backend server validates the signature and checks claims (like expiration) locally without needing a database lookup or centralized session store.

Pros and Cons in Backend Systems

Pros

- **Stateless and scalable:** The server does not need to store session data in memory or a shared database like Redis; all needed info lives inside the token.
- **Cross-domain / microservices:** Easily verified across multiple independent services or APIs that share the same signing key or public key (JWKS).
- **Self-contained:** Carries user identity and roles directly within the payload.

Cons

- **Hard to revoke:** Because JWTs are self-contained, once issued, they remain valid until they expire. Revocation requires workarounds like token blacklists or short-lived access tokens paired with refresh tokens.
- **Size:** They are larger than a tiny opaque session ID because they carry encoded claims.
- **Payload exposure:** Sensitive data (like passwords or PII) must never be put in the payload because it can be easily decoded by anyone holding the token.

For a deeper dive into the specification and security guidelines, see an official JSON Web Token introduction or a practical walkthrough.